

Nonparametric SLR - Lab Activity - Example Solution

B.A. Hartlaub and A.S. Wagaman

Examples for Nonparametric Simple Linear Regression (includes parametric regression comparison)

Highway Runoff and Rainfall

An article in the 1998 Journal of Environmental Engineering studied highway runoff in Austin, Texas by looking at the relationship between rainfall volume (in cubic meters) and runoff volume (also in cubic meters) for different locations. The dataset is loaded next.

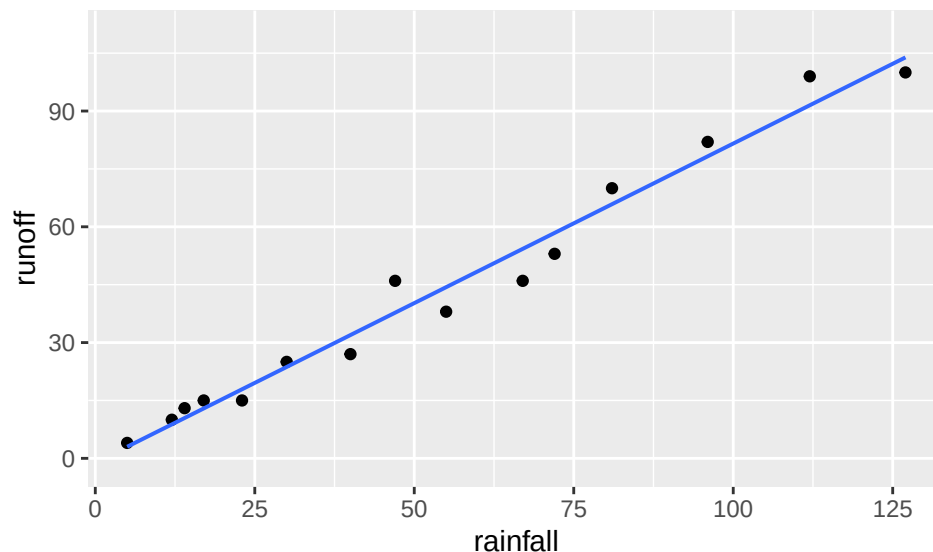
```
rainrun <-  
  read.table("https://awagaman.people.amherst.edu/nonparametricstats/rainrun.txt",  
            h = T)  
summary(rainrun)
```

```
##      rainfall      runoff  
## Min.   : 5.0    Min.   : 4.00  
## 1st Qu.: 20.0   1st Qu.: 15.00  
## Median : 47.0   Median : 38.00  
## Mean   : 53.2   Mean    : 42.87  
## 3rd Qu.: 76.5   3rd Qu.: 61.50  
## Max.   :127.0   Max.    :100.00
```

```
dim(rainrun)
```

```
## [1] 15  2
```

```
ggplot(data = rainrun, aes(x = rainfall, y = runoff)) +  
  geom_point() +  
  geom_lm() # fits parametric regression line and shows points
```



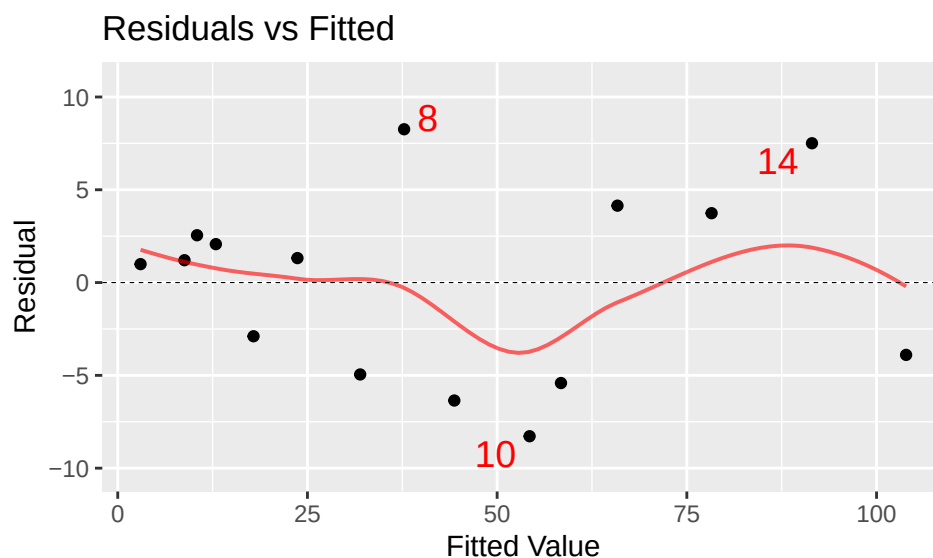
First, we'll fit the usual parametric linear regression model.

```
mod1 <- lm(runoff ~ rainfall, data = rainrun) # fit model Y~X
msummary(mod1) # model output
```

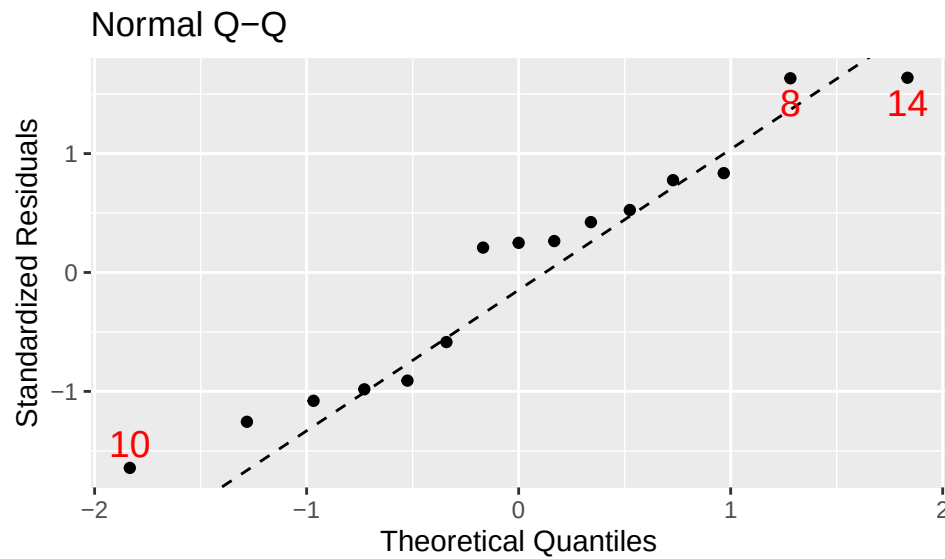
```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.12830    2.36778  -0.477   0.642
## rainfall      0.82697    0.03652  22.642 7.9e-12 ***
##
## Residual standard error: 5.24 on 13 degrees of freedom
## Multiple R-squared:  0.9753, Adjusted R-squared:  0.9734
## F-statistic: 512.7 on 1 and 13 DF,  p-value: 7.896e-12
```

```
mplot(mod1, which = 1) # residual plot to check assumption of constant variance
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
mplot(mod1, which = 2) # qq plot to check normality of residuals (required for errors)
```



What is the equation of the fitted parametric regression line?

SOLUTION:

The line is $\text{predrunoff} = -1.12830 + 0.82697 * \text{rainfall}$.

Do you see any major issues with the assumptions for the parametric test here?

SOLUTION:

The QQ plot looks roughly linear, though there are some departures from this overall linear trend. The residuals plot might suggest increasing variance (issues with constant variance), but it's hard to tell with 15 observations.

Is there a significant linear relationship between rainfall and runoff volume?

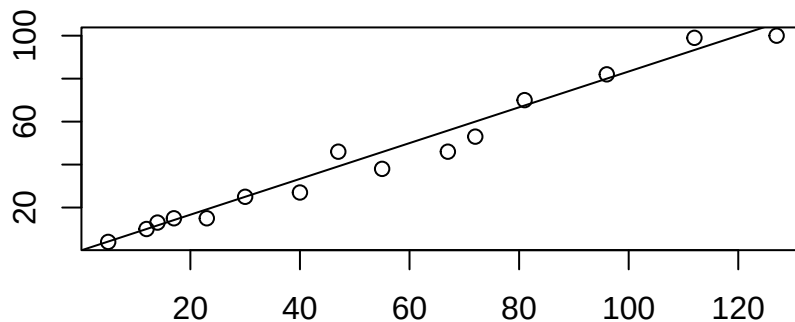
SOLUTION:

The test statistic for the slope is $t=22.642$, and the corresponding p-value is tiny. Yes, the linear relationship appears to be statistically significant.

Now, compare the results to what you find using a nonparametric linear regression model. You will need to provide the code to fit the model.

SOLUTION:

```
# use class examples to help!
# note, increasing r to 6 still leaves the intercept estimate at 0
with(rainrun, theil(rainfall, runoff, beta.0 = 0, type = "t"))
```



```
## Alternative: beta not equal to 0
## C = 101, C.bar = 0.962, P = 0
## beta.hat = 0.833
## alpha.hat = 0
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.705, 0.89
```

What is the equation of the fitted nonparametric regression line?

SOLUTION:

Predicted Median Runoff = $0 + 0.833 * \text{rainfall}$.

Is there a significant linear relationship between rainfall and runoff volume based on the nonparametric procedure? Is this result consistent with the parametric test results?

SOLUTION:

The linear relationship does appear to be statistically significant again. The p-value is approximately 0 (to 4 decimal places).

```
2*pKendall(-0.962, N = 15) # manually compute p-value
```

```
## [1] 8.379367e-07
```

Predict the runoff (response) when the rainfall value is 42, using both models. How should you interpret the predicted values? (There should be a difference.)

SOLUTION:

```
mod1pred<- -1.12830 + 0.82697*42; mod1pred # parametric
```

```
## [1] 33.60444
```

```
mod2pred<- 0 + 0.833*42; mod2pred # nonparametric
```

```
## [1] 34.986
```

The parametric model says the average runoff expected when rainfall = 42 is 33.60444 cubic meters. The nonparametric model says the *median* runoff expected when rainfall = 42 is 34.986 cubic meters. 42 is in the range of observed X values so this is not extrapolation.

Lichen

An article in Environmental Monitoring and Assessment in 1993 explored the relationship between NO3 wet deposition (grams per cubic meter) and lichen (percent dry weight) in order to try to predict lichen dryness using NO3. The dataset is loaded next.

```
lichen <-  
  read.table("https://awagaman.people.amherst.edu/nonparametricstats/lichen.txt",  
            h = T)  
summary(lichen)
```

```
##          no3          lichen  
## Min.    :0.0500   Min.    :0.4800  
## 1st Qu.:0.1200   1st Qu.:0.5200  
## Median :0.4200   Median :0.8600  
## Mean    :0.4554   Mean    :0.8054  
## 3rd Qu.:0.6800   3rd Qu.:1.0000  
## Max.    :0.9200   Max.    :1.7000
```

```
dim(lichen)
```

```
## [1] 13  2
```

If the goal is to predict lichen dryness using NO3, which variable should be the response and which the predictor?

SOLUTION:

NO3 is the predictor and lichen dryness is the response.

Fit a parametric regression model. Then address the questions below.

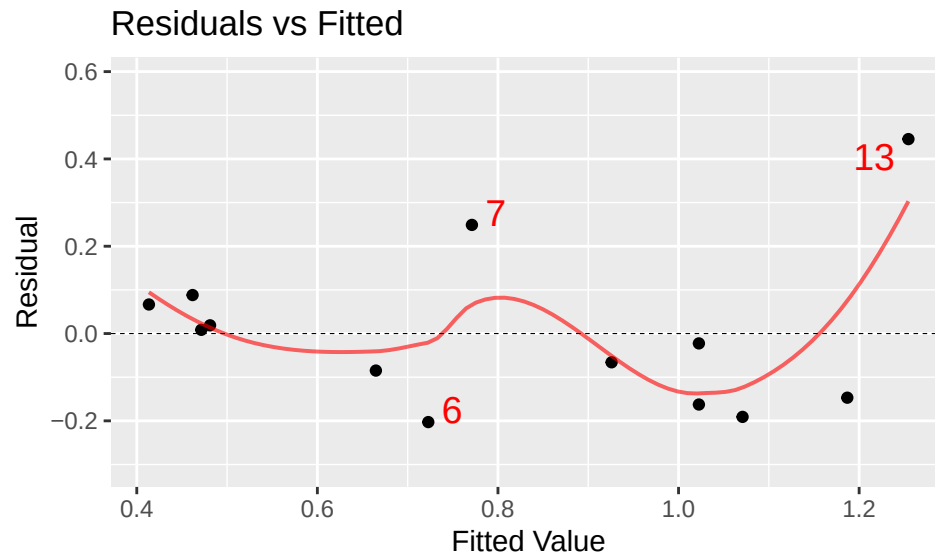
SOLUTION:

```
mod2 <- lm(lichen ~ no3, data = lichen)  
msummary(mod2)
```

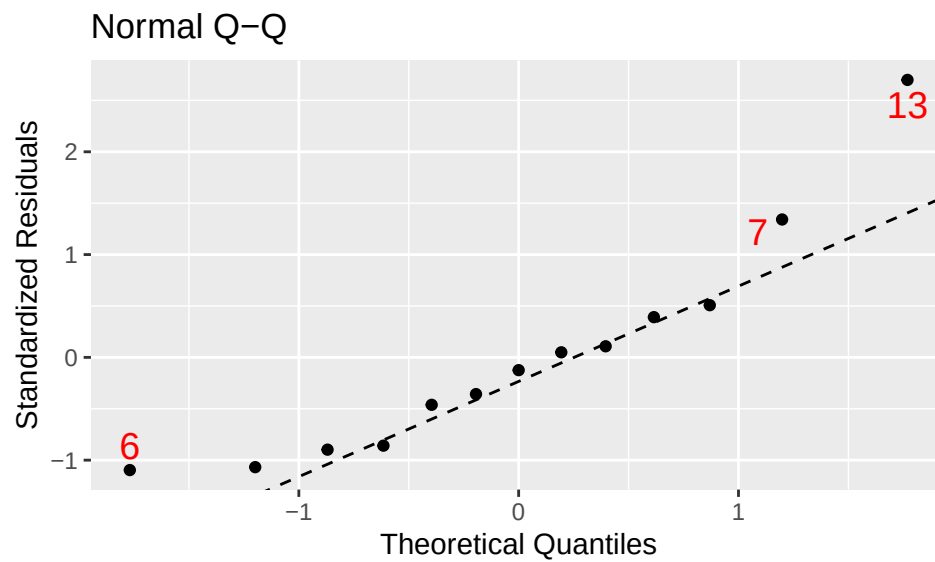
```
##          Estimate Std. Error t value Pr(>|t|)  
## (Intercept)  0.36510    0.09904   3.686 0.003586 **  
## no3          0.96683    0.18292   5.286 0.000258 ***  
##  
## Residual standard error: 0.1932 on 11 degrees of freedom  
## Multiple R-squared:  0.7175, Adjusted R-squared:  0.6918  
## F-statistic: 27.94 on 1 and 11 DF,  p-value: 0.0002581
```

```
mplot(mod2, which = 1)
```

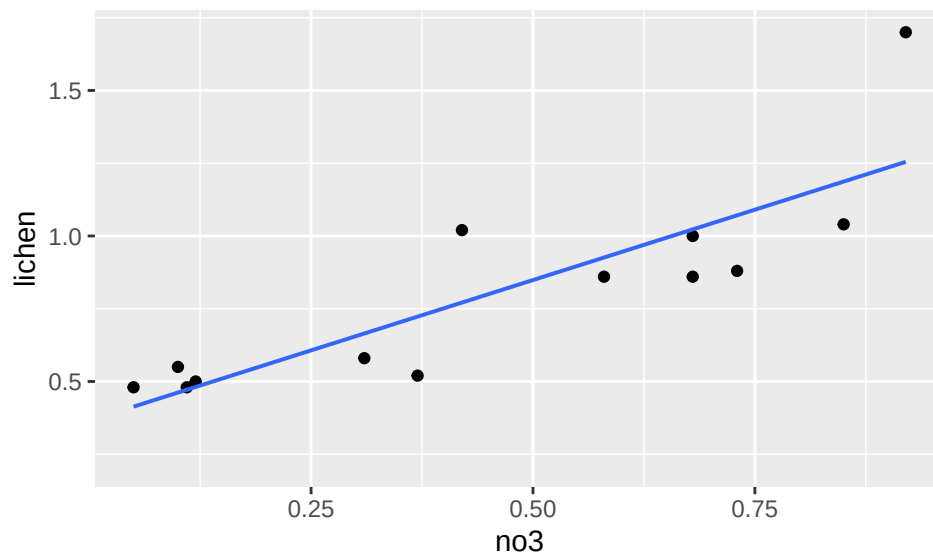
```
## `geom_smooth()` using formula = 'y ~ x'
```



```
mpplot(mod2, which = 2)
```



```
ggplot(data = lichen, aes(x = no3, y = lichen)) +  
  geom_point() + geom_lm()
```



What is the equation of the fitted parametric regression line?

SOLUTION:

`pred_LichenDryness= 0.36510 + 0.96683*NO3`

Do you see any major issues with the assumptions for the parametric test here?

SOLUTION:

The residual QQ plot shows some outliers - further from the line than we'd like. The residuals versus fitted plot shows some possibility of increasing variance, but definitely shows the outliers. Thus, we would be concerned about the conditions due to the outliers - observations 7 and 13 in particular on the high end, and 6 on the low end.

Is there a significant linear relationship between the two variables?

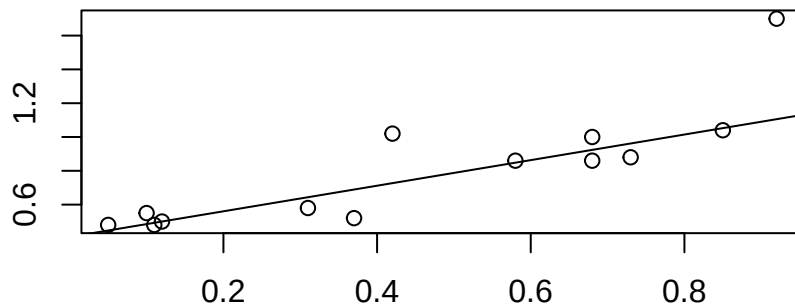
SOLUTION:

If we assume the conditions hold, then yes. We have a t value of 5.286 and p-value = 0.000258. This indicates the linear relationship is statistically significant.

Now, compare the results to what you find using a nonparametric linear regression model. Fit the model first.

SOLUTION:

```
with(lichen, theil(no3, lichen, beta.0 = 0, type = "t", r = 6))
```



```
## Alternative: beta not equal to 0
## C = 58, C.bar = 0.74359, P = 0.000123
## beta.hat = 0.756757
## alpha.hat = 0.409189
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.5, 1.4
2*pKendall(-0.74359, N = 13) # manual p-value computation to check

## [1] 0.0001234566
```

What is the equation of the fitted nonparametric regression line?

SOLUTION:

pred_MedianLichenDryness= 0.409 + 0.757*NO3

Is there a significant linear relationship between the two variables based on the nonparametric procedure? Is this result consistent with the parametric test results?

SOLUTION:

The p-value is 0.000123 (we increased r above so it wasn't rounded to 0), so yes, there is a significant linear relationship based on the nonparametric test. This is consistent with the parametric test results.

Would it be appropriate to predict the NO3 value when the lichen percentage is 50 percent using the regression lines obtained above? Explain.

SOLUTION:

No. The regression line we have uses NO3 to predict lichen dryness. We would need to refit the regression line to do the reverse. The slopes are NOT simply inverses of each other (common misconception encountered in intro stats), so you can't just reverse the lines this way.

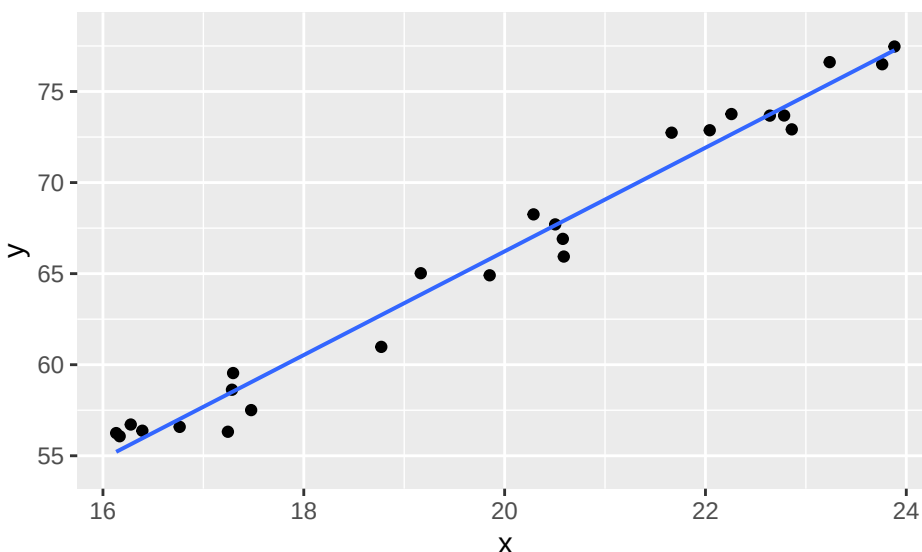
Comparing Parametric and Nonparametric SLR via Simulation

We want to investigate settings where the differences between the nonparametric and parametric procedures can be observed. This lies in studying the distribution of errors, which we can explore via simulation.

Let's consider a model where $Y = 3X + 6 + \text{error}$, and then tinker with what distribution we use for the error. Remember that the errors are supposed to have median 0, but we can change the spread and shape of the distribution quite a bit and still satisfy this. Most of our examples below also involve symmetric distributions, but that doesn't have to be the case.

Example 1 - Uniform Errors

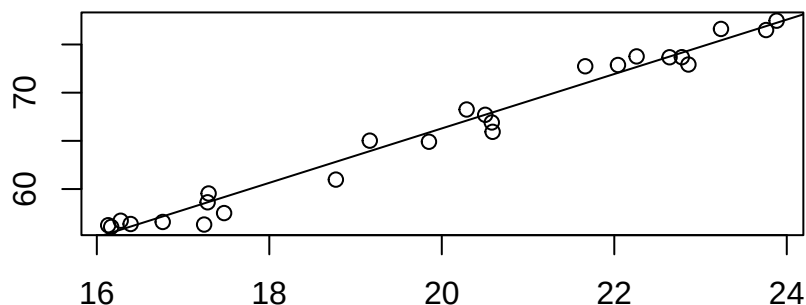
```
set.seed(52) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + runif(25, -2, 2) # Y=3X+6+ error; error is uniform(-2,2)
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  9.31150    1.77586   5.243 2.56e-05 ***
## x           2.84558    0.08875  32.063 < 2e-16 ***
##
## Residual standard error: 1.167 on 23 degrees of freedom
## Multiple R-squared:  0.9781, Adjusted R-squared:  0.9772
## F-statistic: 1028 on 1 and 23 DF, p-value: < 2.2e-16
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric fit
```



```
## Alternative: beta not equal to 0
## C = 258, C.bar = 0.86, P = 0
## beta.hat = 2.823
## alpha.hat = 9.838
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 2.623, 2.994
```

How do the parametric and nonparametric lines compare here? (Look at both plots and their coefficients!) Is this what you expected?

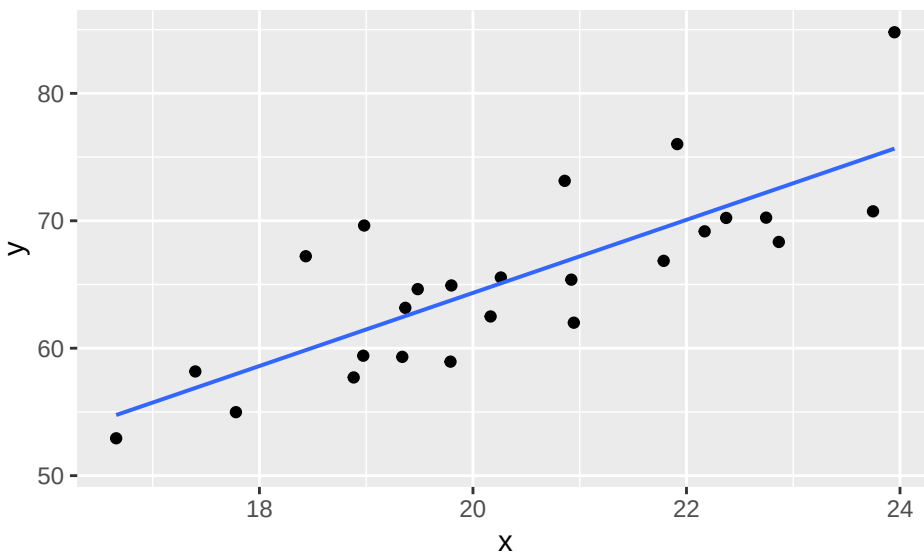
SOLUTION:

Remember that your responses may differ some if you change the seed, as the data set changes! Answers below are for the seeds provided.

We know that the true line is $Y=3X+6$. Both methods yield a slope near, but slightly under 3, and intercepts that are too large (both are above 9, instead of being close to 6). Overall the methods look comparable. Yes, this is to be expected. The error is small, and while non-normal, is not too heavy in the tails yet. The parametric method should work well and nonparametric method shouldn't underperform either.

In the code below, increase the spread of the errors by using a wider range for the uniform distribution (like -5 to 5, etc.)

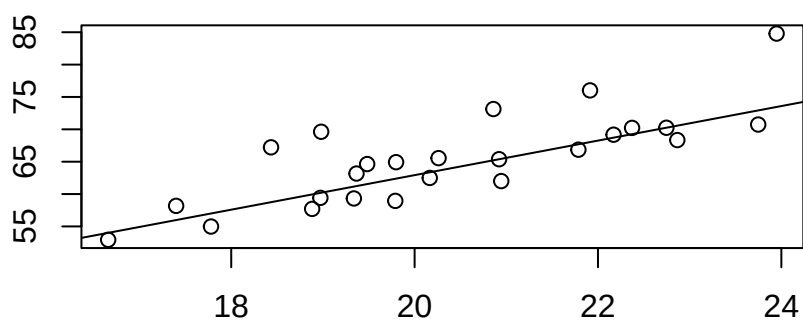
```
set.seed(22) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + runif(25, -7, 7) # change -2 and 2 to wider range, still centered at 0
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.9760     9.3125   0.749   0.461
## x             2.8682     0.4549   6.306 1.96e-06 ***
##
## Residual standard error: 4.347 on 23 degrees of freedom
## Multiple R-squared:  0.6335, Adjusted R-squared:  0.6176
## F-statistic: 39.76 on 1 and 23 DF,  p-value: 1.961e-06
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric fit
```



```
## Alternative: beta not equal to 0
## C = 186, C.bar = 0.62, P = 0
## beta.hat = 2.668
## alpha.hat = 9.563
```

```
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 1.897, 3.505
```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

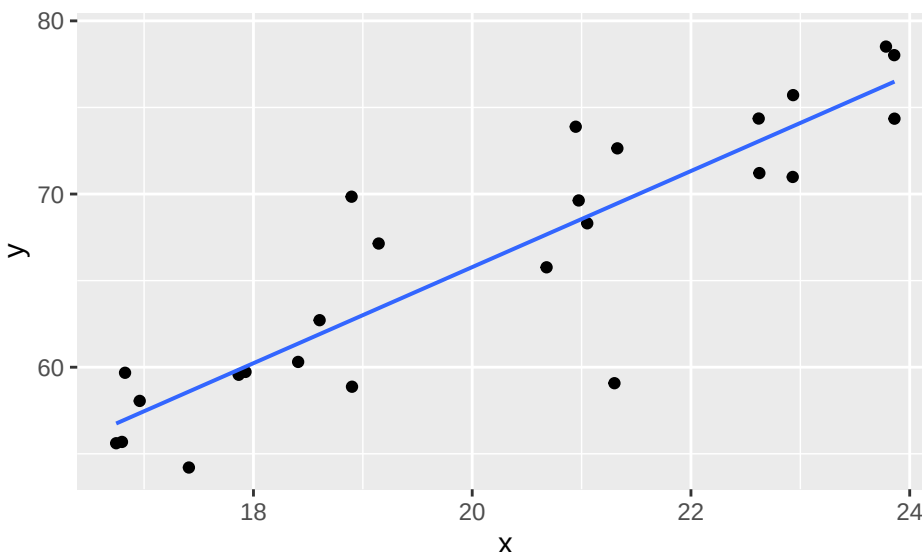
The nonparametric method actually seems a little further off now (We tried several values, consistency was noted) than the parametric method. The slopes are still not radically different from 3 though. Maybe a little unexpected here, but the nonparametric method does not always do as well as we might like with uniform errors compared to the parametric method.

Note that this is only a single data set comparison (under each distribution of errors). To do a more appropriate comparison, you'd need to do multiple runs here, comparing multiple slope coefficients across both models (or figure out some other measure you want to use for assessment).

Example 2 - Double Exponential Errors

We've seen before that the parametric and nonparametric procedures are often near equivalent in settings with either underlying normal distributions or uniform distributions, so let's get a little bit more extreme.

```
set.seed(29) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rlaplace(25, 0, 2) # Need center at 0, can change scale parameter
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

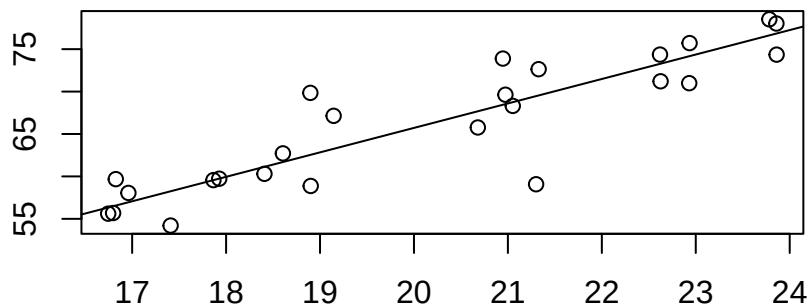
```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  10.2940     6.0012   1.715  0.0997 .
## x             2.7744     0.2959   9.375 2.55e-09 ***
##
## Residual standard error: 3.562 on 23 degrees of freedom
## Multiple R-squared:  0.7926, Adjusted R-squared:  0.7836
## F-statistic: 87.89 on 1 and 23 DF, p-value: 2.548e-09
```

```

confint(fm)

##              2.5 %      97.5 %
## (Intercept) -2.120397 22.708328
## x           2.162173  3.386567
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output

```



```

## Alternative: beta not equal to 0
## C = 212, C.bar = 0.707, P = 0
## beta.hat = 2.883
## alpha.hat = 8.065
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 2.286, 3.3

```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

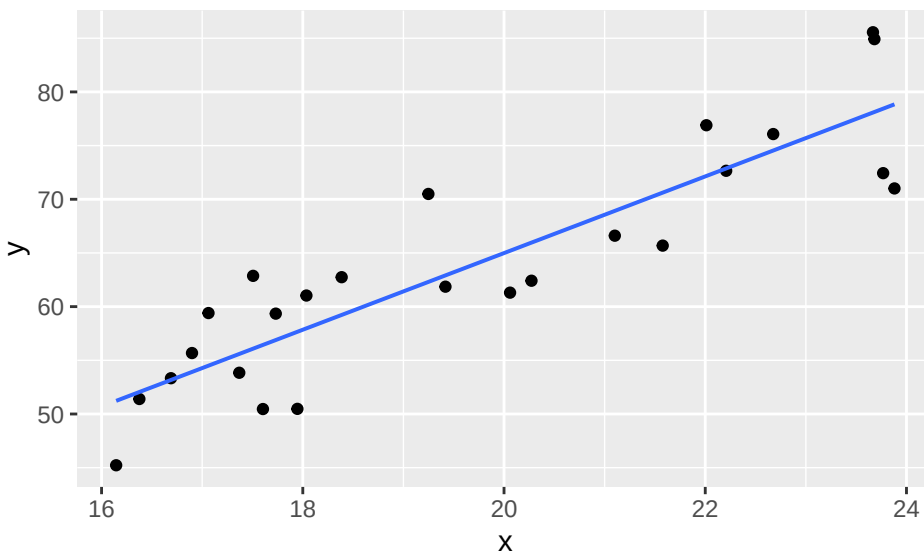
The line is still $Y=3X+6$, but we have double exponential errors. The nonparametric slope is now closer to 3 than the parametric slope. Both still underestimate the slope, so their intercepts are also too high as a result. The CIs indicate we can't reject 3 as a reasonable slope value.

Once again, up the spread of the errors to see what happens. In the code below, increase the spread of the errors by using a larger value for the scale parameter

```

set.seed(51) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rlaplace(25, 0, 5) # Change scale parameter to be > 2
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit

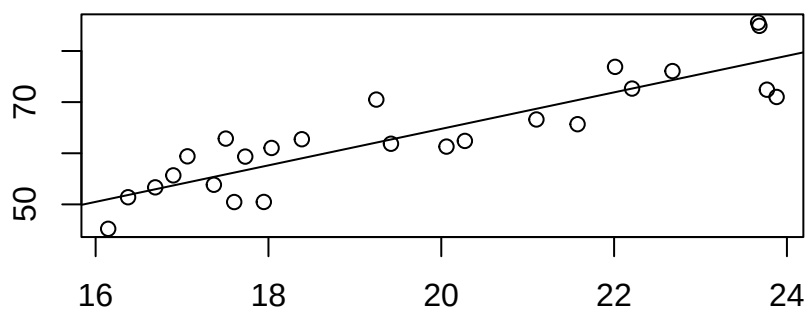
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -6.413      7.710  -0.832   0.414
## x              3.570      0.389   9.177 3.76e-09 ***
##
## Residual standard error: 5.011 on 23 degrees of freedom
## Multiple R-squared:  0.7855, Adjusted R-squared:  0.7762
## F-statistic: 84.22 on 1 and 23 DF,  p-value: 3.764e-09
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 208, C.bar = 0.693, P = 0
## beta.hat = 3.57
## alpha.hat = -6.624
```

```
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 2.575, 4.619
```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

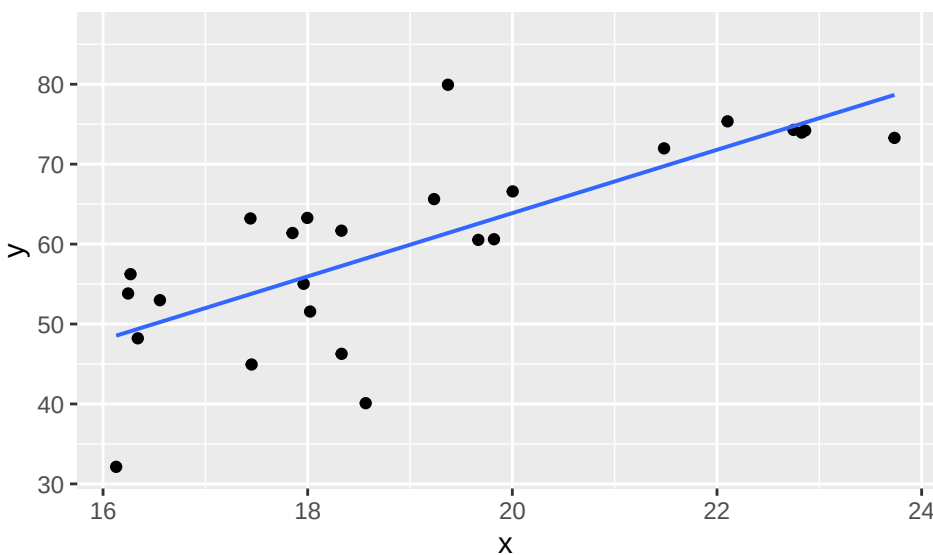
For many different values of the scale parameter for the errors, we see the slope estimates being similar (identical, even) for both the parametric and nonparametric methods. The intercepts though, continue to differ. This is slightly unexpected, we haven't really caused issues with the parametric line yet.

Note that if the error variance starts getting too large, both lines will start to have issues - the relationship is weaker.

Example 3 - Cauchy Errors

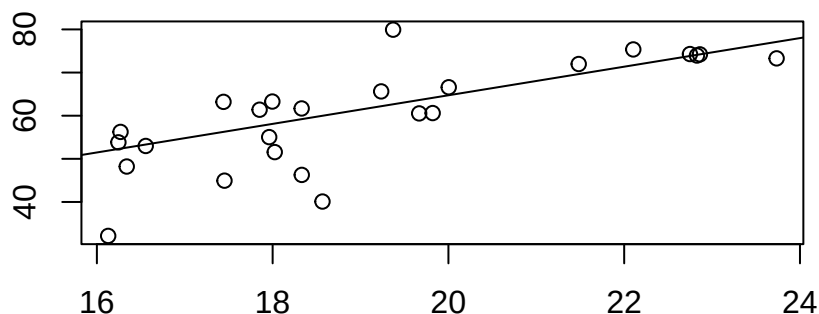
One last example before we look at the effect of outliers, though this example helps to illustrate that too. What happens if we use a Cauchy distribution for the errors?

```
set.seed(87) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rcauchy(25, 0, 2) # Need center at 0, can change scale parameter
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -15.332     13.730  -1.117   0.276
## x              3.960       0.714   5.547 1.21e-05 ***
##
## Residual standard error: 8.157 on 23 degrees of freedom
## Multiple R-squared:  0.5722, Adjusted R-squared:  0.5536
## F-statistic: 30.77 on 1 and 23 DF,  p-value: 1.214e-05
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 156, C.bar = 0.52, P = 0
## beta.hat = 3.314
## alpha.hat = -1.558
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 2.335, 4.812
```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

The Cauchy distribution is one of the more extreme error distributions, so we expect to see issues with the parametric method here. The slopes show this fairly immediately. The parametric slope estimate is almost 4, compared to the nonparametric estimate which is still fairly close to 3 (3.3).

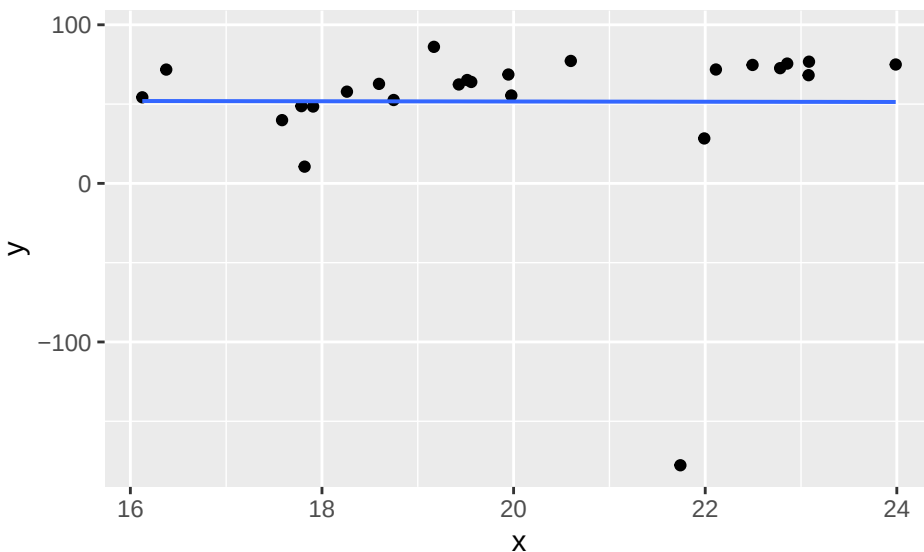
Are there outliers here? What is affecting the performance of the parametric regression line?

SOLUTION:

Based on the plots, there are a few points now fairly far from the regression line which is affecting the overall fit (pulling the line towards them), but that impacts the parametric line more than the nonparametric line.

One last time, up the spread of the errors to see what happens. In the code below, increase the spread of the errors by using a larger value for the scale parameter. Again, if it goes too large, the relationship can become weak enough that both methods will start to have trouble.

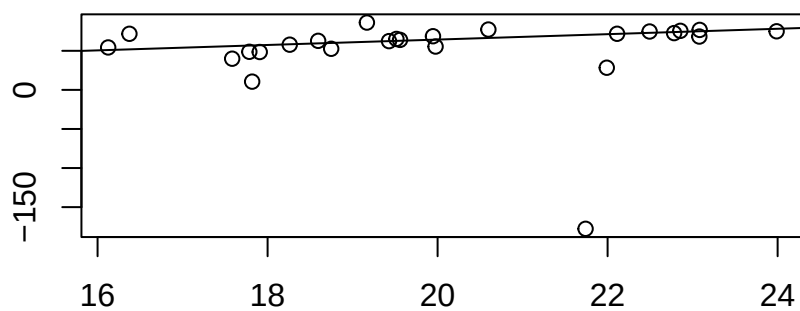
```
set.seed(34) # you can change this!
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rcauchy(25, 0, 6) # change scale parameter > 2
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```

```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) 53.15701   93.48696   0.569   0.575
## x          -0.07583    4.63161  -0.016   0.987
##
## Residual standard error: 51.74 on 23 degrees of freedom
## Multiple R-squared:  1.165e-05, Adjusted R-squared:  -0.04347
## F-statistic: 0.0002681 on 1 and 23 DF,  p-value: 0.9871
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 120, C.bar = 0.4, P = 0.005
## beta.hat = 3.456
## alpha.hat = -4.763
```

```
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 1.603, 5.516
```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

We can see major differences here. The parametric slope estimate is NEGATIVE, and not significantly different from 0. The nonparametric estimate is 3.456, so a little high, but the CI shows that three is a reasonable value for the slope. Here, you'd want the nonparametric procedure. This is as expected given the underlying Cauchy distribution for the error terms.

Are there outliers here? What is affecting the performance of the parametric regression line?

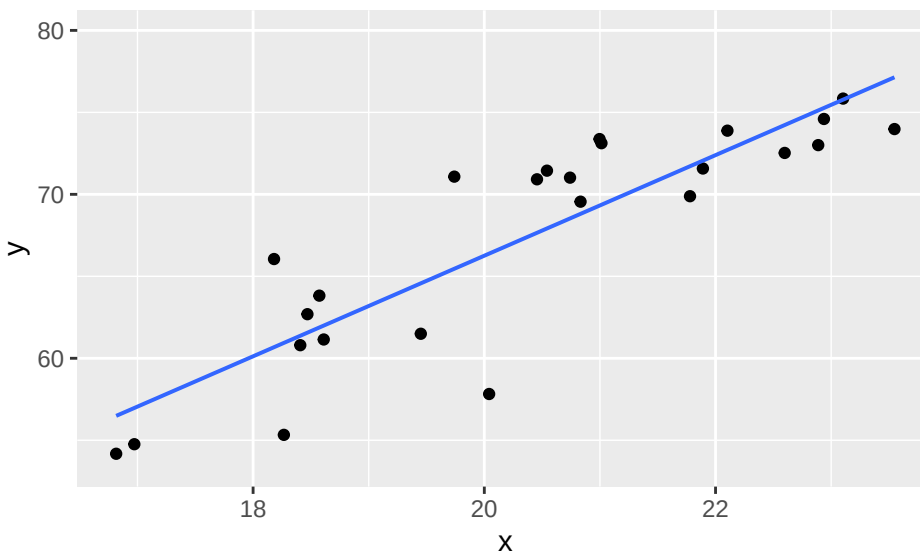
SOLUTION:

There is a clear outlier that has an x of almost 22 but a very large magnitude negative y value due to the errors which is heavily influencing the parametric line.

Example 4 - Effect of Outliers

The Cauchy examples should have started to suggest that the parametric procedures are much more sensitive to outliers than the nonparametric procedures. Here, we investigate that more directly. We want to start with a relationship where both procedures do fairly well. We can even use a normal distribution for the errors.

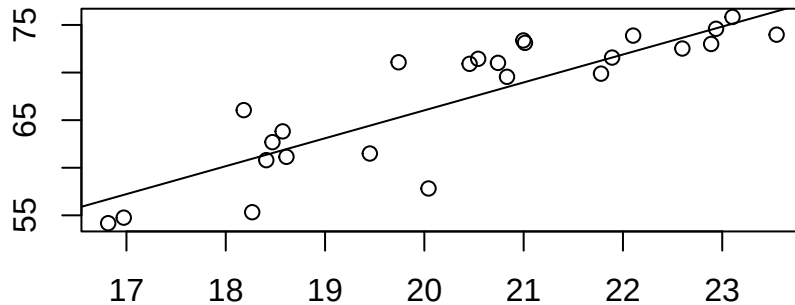
```
set.seed(76) # keep this, or remember to change both here and below
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rnorm(25, 0, 3) # Y=3X+6+ error; error is N(0,3)
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   4.904      7.341    0.668   0.511
## x             3.068      0.359    8.544 1.36e-08 ***
##
## Residual standard error: 3.452 on 23 degrees of freedom
```

```
## Multiple R-squared:  0.7604, Adjusted R-squared:  0.75
## F-statistic: 73.01 on 1 and 23 DF,  p-value: 1.361e-08
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 218, C.bar = 0.727, P = 0
## beta.hat = 2.933
## alpha.hat = 7.375
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 1.911, 3.824
```

How do the parametric and nonparametric lines compare here? Is this what you expected?

SOLUTION:

Both slope values are very near to 3. The intercepts are both off, but not by much. The methods are doing well, and the nonparametric one is not suffering too much in this setting either.

P.S. It should be good that the nonparametric line is not doing badly in a setting where the parametric line is expected to do well.

Now, let's create an influential point by modifying an existing data point. This is a point that we think will affect the regression line strongly because it is off the general pattern.

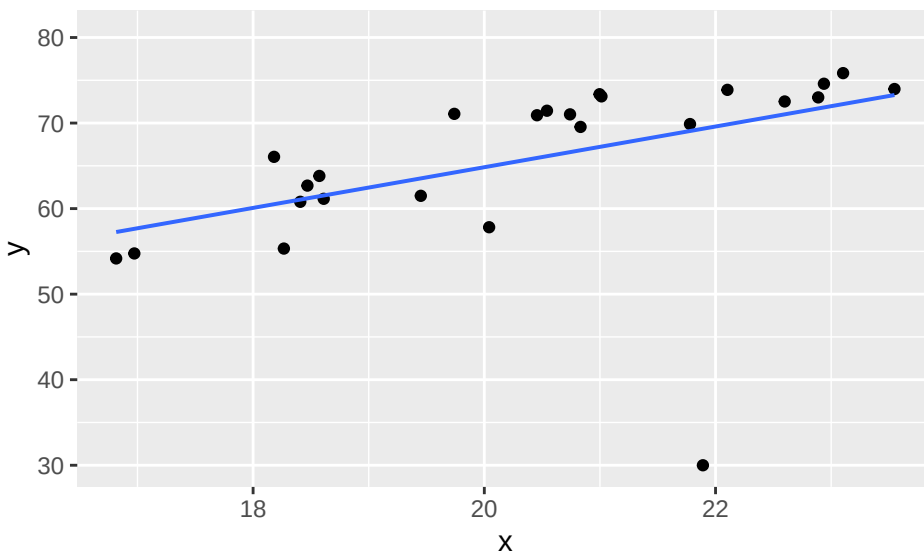
The first and second observations were:

```
head(mydata, 2)
```

```
##      x      y
## 1 20.99668 73.37156
## 2 21.89073 71.57486
```

The second observation is very close to both lines, so let's tinker with that one. We will leave it at that X value, but change the y value. Instead of 71.57, let's drop the Y value to 30, and see what happens.

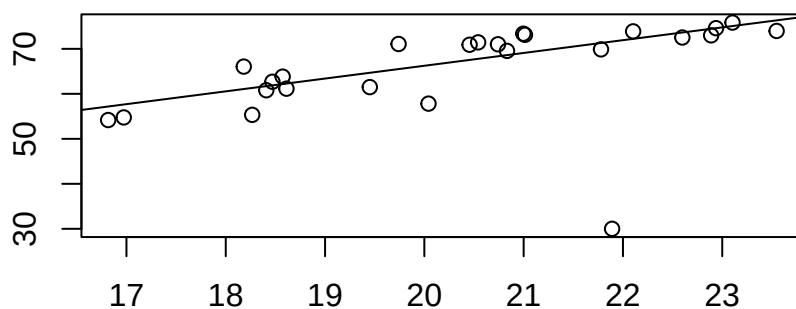
```
# brute force way to change point
mydata[2, 2] <- 30
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  17.2657    19.4769   0.886  0.3845
## x           2.3787     0.9525   2.497  0.0201 *
##
## Residual standard error: 9.159 on 23 degrees of freedom
## Multiple R-squared:  0.2133, Adjusted R-squared:  0.1791
## F-statistic: 6.237 on 1 and 23 DF,  p-value: 0.02011
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 186, C.bar = 0.62, P = 0
## beta.hat = 2.845
## alpha.hat = 9.347
```

```
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 1.683, 3.828
```

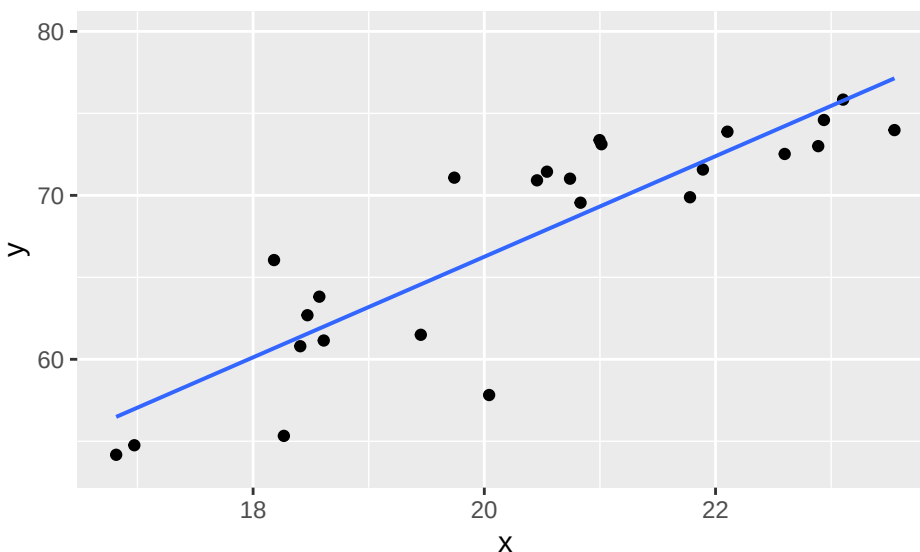
What effect did you observe in the regression lines? Which line was more affected by the unusual point?

SOLUTION:

The slopes of both regression lines have dropped now, but the parametric one was much more affected by the unusual point.

Points further from the mean of the x's have more leverage to influence the regression line. Let's reload the data and tinker with one of the furthest points out.

```
set.seed(76) # match what you started Cauchy section with
x <- runif(25, 16, 24) # 25 obs from a Unif(16,24)
y <- 3*x + 6 + rnorm(25, 0, 3) # Y=3X+6+ error; error is N(0,3)
mydata <- data.frame(x, y)
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```

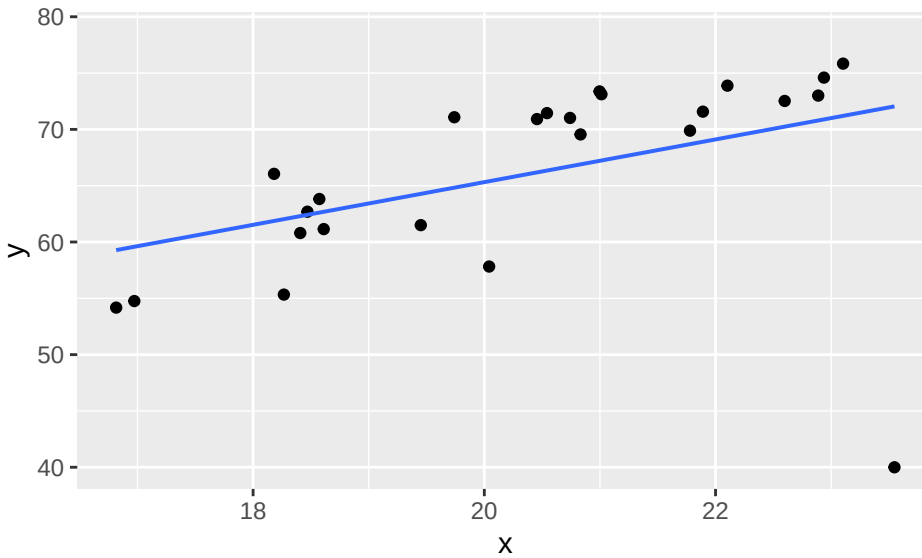


```
max(x)
## [1] 23.54757
which(x > 23.545)
## [1] 13
mydata[13, ]
```

```
##           x           y
## 13 23.54757 73.98012
```

The maximum x value (we could also do minimum) is observation 13. Let's change (drop) its Y value to 40 and see what happens.

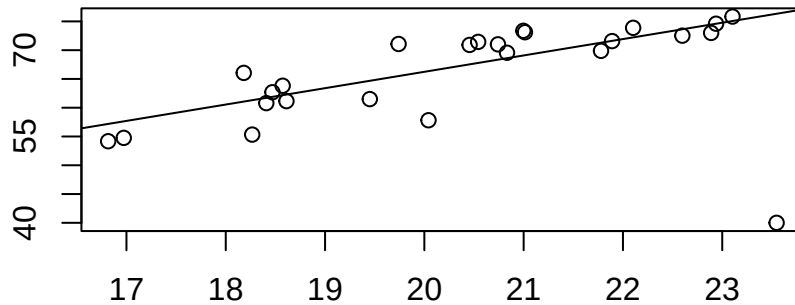
```
mydata[13, 2] <- 40
ggplot(mydata, aes(x = x, y = y)) + geom_point() + geom_lm() # parametric fit
```



```
fm <- lm(y ~ x, data = mydata)
msummary(fm)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  27.4015    17.0013   1.612  0.1207
## x            1.8957     0.8314   2.280  0.0322 *
##
## Residual standard error: 7.995 on 23 degrees of freedom
## Multiple R-squared:  0.1844, Adjusted R-squared:  0.1489
## F-statistic: 5.199 on 1 and 23 DF,  p-value: 0.03219
```

```
with(mydata, theil(x, y, beta.0 = 0, type = "t")) # nonparametric plot is part of output
```



```
## Alternative: beta not equal to 0
## C = 174, C.bar = 0.58, P = 0
## beta.hat = 2.845
## alpha.hat = 9.347
```

```
##  
## 1 - alpha = 0.95 two-sided CI for beta:  
## 1.644, 3.811
```

What effect did you observe in the regression lines? Which line was more affected by the unusual point?

SOLUTION:

Here, the slopes dropped again but the parametric one dropped to about 1.9 instead of 2.8 like the nonparametric line. This again indicates the parametric line is more sensitive to outliers.

What if we dropped the y-value even further there to something like 20? Which line would you expect to be more affected?

SOLUTION:

The parametric line would be most affected.

Feel free to check to verify your thoughts.

You should be able to describe the benefits of nonparametric procedures based on these examples, as well as understand a bit more about how the parametric procedures can be affected by unusual points or non-normal errors.

Data Sources

Barrett, Michael E., et al. "Characterization of highway runoff in Austin, Texas, area." *Journal of Environmental Engineering* 124.2 (1998): 131-137.

Bruteig, Inga E. "The epiphytic lichen *Hypogymnia physodes* as a biomonitor of atmospheric nitrogen and sulphur deposition in Norway." *Environmental monitoring and assessment* 26.1 (1993): 27-47.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to *Nonparametric Statistical Methods* (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.